Context Compounds, Capability Does Not: An Experience Report on a Persistent Multi-Agent System

Kyle Fox
Independent Researcher
Austin, Texas, USA
Email: *[to fill at submission]*

***Abstract*—Conductor is a fleet of persistent, memory-divergent agent sessions — one base model, durable per-session context, a shared append-only bus, and a human gating only irreversible actions. Over a months-long live deployment it grew to 35 concurrent projects while human touches per project fell by half; the human originated under 1% of coordination traffic while the sessions co-designed and hardened their own coordination substrate in vivo under adversarial peer review — closing ∼20 named failure modes — and caught one another's defects from divergent empirical vantages. Our central finding: what compounds across tasks is the *committed carrier* — the code, asserts, and checked-in docs any fresh session can read — not the session's lived context. Context buys *efficiency and recognition* — 2.6× wall-clock and 36% fewer steps on the pre-registered task — and, on some tasks, a measurable *thoroughness regression* (pattern-match-and-stop). We establish this by testing the intuition such designs rest on — that accumulated context makes a session more capable on task $N{+}1$ because of tasks $1 \ldots N$ — under six A/B ablations and a pre-registered baseline: every fresh session holding the carrier re-derived every fix; memory changed the path cost, never the outcome. The practitioner consequence: run context-heavy sessions as the default workforce, routing work by demonstrated history; commit everything that must stay true to the carrier; and audit long-lived sessions with periodic fresh-eyed clones.**

***Index Terms*—LLM agents, multi-agent systems, agent memory, coordination substrate, human-in-the-loop, experience report, ablation study, self-adaptive systems**

I. Introduction and Contributions

THE prevailing agentic pattern is an orchestrator dispatching to stateless workers — reproducible, but *permanently at task 1*: no carried expertise, no division of labour, no shared context. Conductor inverts the premise: long-lived sessions with divergent memories talk to each other and build their own coordination substrate. Our contributions, stated at the level the evidence supports:

1) A method — in-vivo adversarial co-design under reversibility-calibrated HITL. The substrate's primitives were not designed up front; they were *provoked by silent-loss failures* the fleet hit in operation, proposed and attacked by peers on the bus, and gated by a human whose approval is scoped to *irreversibility* (commits are free; pushes, and anything that outlives the session, are gated). We present it as a *replicable process with a stated mechanism and an existence proof* — it produced and hardened a working substrate — not as a result validated against the alternative (an engineer designing the primitives up front); we did not run that comparison (§I-A).
2) A system implementing it (bus, verified orders, a project layer, a decision shield, measured governance), described as an experience report with the mechanical record.
3) An evaluation that separates what the deployment *establishes* from what it only *suggests*, and an account of the method's own failure modes — including a circularity we fell into and were corrected on.

This is an experience report, and we ask to be judged at that bar: the contribution is what a months-long real deployment reveals — we claim neither statistical generality ($N{=}1$ deployment) nor emergent intelligence.

Summary of findings (§V): *context compounds; capability does not. What compounded* (confirmed by the record): (1) stable roles and a load-shared division of labour; (2) a fleet-designed, fleet-hardened coordination substrate — the reusable method; (3) cross-context peer review catching defects individual sessions and the human missed; and (4) real efficiency from lived context. *What did not*: task-solving capability — under six A/B ablations and the pre-registered baseline it resides in the committed carrier, not the session.

A. What we set out to show, and what survived

The contributions are not epistemically equal. The compounding-competence hypothesis — a context-heavy peer substrate cheaper on task $N{+}1$ because of tasks $1 \ldots N$ — **did not survive** (§V-D, §V-E): what compounds is the committed carrier, the paper's most solid *evaluated* result. The method (contribution 1), by contrast, has *no controlled evidence*: we did not run in-vivo adversarial co-design head-to-head against an engineer specifying the primitives up front, so we do not claim it is superior — only that it is a replicable process with a described mechanism that demonstrably *produced* a working, hardened substrate. Its falsifier: the method claim weakens if the failures the adversarial process caught would have been caught as readily by ordinary single-author review, or if the primitives it converged on were worse than a competent up-front design. One *observational* signal is consistent with it (not

TABLE I

HUMAN TOUCH PER ACTIVE PROJECT (`HISTORY.JSONL`). JAN/MAR (1 AND 7 PROMPTS) ARE PRE-RAMP STARTUP AND APRIL THE SUBSTRATE'S BUILD-OUT MONTH, ALL BOOTSTRAP-HEAVY AND OMITTED. THREE BUCKETS CANNOT ESTABLISH A TREND AND JUNE ≈ MAY; WE REPORT THE SHAPE (NEAR 70 MAY–JUNE, THEN 36 IN JULY), CORROBORATED BY A 14-WEEK BREAKDOWN (REPLICATION PACKAGE). ANALYZED IN §V-A.

Month	Human prompts	Active projects	Prompts / project
2026-04	2,027	10	202.7
2026-05	1,117	16	69.8
2026-06	1,936	27	71.7
2026-07	1,270	35	36.3

a controlled test): human prompts per active project declined (Table I) — from near 70 across May–June to 36 in July (monthly; a finer weekly series in §V-A shows the same direction) — *as* concurrent projects grew from 16 to 35. The human moves from courier toward architect, which is what the method is for — a trend with the usual longitudinal confounds, not proof.

II. RELATED WORK

(Full treatment: `related-work.md`.) Capability-aware routing exists but is keyed on declared role (AutoGen [1]; MetaGPT [2]; ChatDev [3]; CrewAI, *stateless by default*) or predicted fit (LangGraph supervisor; DyLAN [4]; CaptainAgent [5]; AutoAgents [6]; MasRouter [7]) over agents that are stateless-by-default — with query-level model routers (RouteLLM [8], FrugalGPT [9]) a separate axis, and Mixture-of-Agents [10] an ensemble that aggregates all proposers, not a router (reclassified per review). Voyager [11] accumulates competence for a *single* agent; ADAS [12] / Darwin Gödel Machine [13] evolve a *population of agent designs* at design-search time; MAPE-K (Kephart & Chess, *Computer* 36(1), 2003) [14] is the classical self-adaptive-control analogue.

Prior art the substrate rediscovers (and we name it as such). Several of Conductor's primitives are not new in isolation — and saying so is evidence *for* the method, not against it: an adversarial process that independently reconverged on known load-bearing designs is weak-but-real evidence the process finds *real* designs. The append-only shared log that peers read and write, with a nominated lead coordinating, is a blackboard architecture (Hearsay-II [15]; the blackboard-for-control model [16]); the verified-order lifecycle (deliver only if the artifact landed) is two-phase commit; the human approval bound to an action token rather than conveyed in prose is capability-based authorization; and the design loop itself rediscovers evolutionary prototyping [17], [18], the spiral model [19], and game-day/chaos-engineering practice [20]. Our contribution is not these mechanisms but the *setting* — long-lived, context-heavy, same-model peers rediscovering and composing them in vivo under a reversibility-gating human.

Honest positioning (we do not overclaim novelty). A 2025 memory-augmented-MAS line does pursue cross-task team competence — most pointedly G-Memory [21] (hierarchical memory "nurturing the progressive evolution of agent teams,"

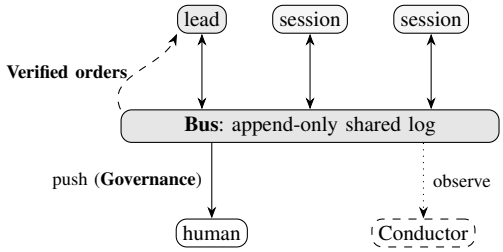

Fig. 1. The composition: persistent *sessions* post to and read an append-only **Bus**; a lead fans work as **Verified orders** (`deliver` reads the artifact back before acknowledging); the *human* gates only irreversible acts (**Governance**: the SHA-pinned push gate); *Conductor* observes read-only and never drives the agents. The through-line: *verify the outcome, not the intention.*

i.e. task $N{+}1$ better because of $1 \ldots N$ across a team), plus RCR-Router [22] (routing *structured memory to agents within a task*) and the transactive-memory framing for agents. So "team competence accumulates" is *not* our novelty. Two things differ. (i) Each peer is a long-lived, human-in-the-loop *session identity* that lived the work — not an ephemeral agent doing retrieval over a shared memory store. We are careful to state what that buys, because our own ablations (§V-D) constrain it: the lived identity does *not* raise outcome *capability* (a fresh session with the same committed carrier reaches the same fixes); what it buys is efficiency and routing quality — fewer steps to a correct fix, and a lead who can route work by a peer's demonstrated history rather than a declared persona. (ii) Routing is therefore a lead's grounded judgment over real artifacts (bus, asset cards, `CLAUDE.md`), keyed on lived history rather than a self-described role. We do *not* list "experience report vs. benchmark evaluation" as a third differentiator: that is a difference in *rigour*, and in the direction of less. We frame our contribution as *under-explored relative to* that memory-augmented line, from which we differ in kind (efficiency-and-routing from *lived* identities, studied as a running deployment) — not as a gap nobody has touched.

III. THE ARCHITECTURE

One base model; each session a persistent identity with durable memory; a read-only observer (Conductor) that never drives the agents (Fig. 1). Primitives, each landed as a versioned mechanism in operation:

- Bus — append-only shared log; directed mail is auto-delivered (idle recipient woken), which is the courier-elimination mechanism (and an *unreliable* one — §V-A).
- Verified orders — `deliver` reads the artifact back and refuses unless it landed.
- Project layer — a nominated lead drafts a job-DAG plan the human approves at one gate, then fans jobs (directed orders) along dependency edges; a *peer*, not the lead, accepts the lead's own deliverables.
- Decision shield — worker decides technical calls and logs them; lead decides project calls; a denylist + severity hatch go straight to the human, and the lead is *structurally barred* from deciding them.

- Governance — SHA-pinned push gate; observer-owned admission throttle (not the lead — fox/henhouse); a measured per-project token meter (estimation is theater; only the live meter gates).

The through-line is verify the outcome, not the intention.

IV. The Method: In-Vivo Adversarial Co-Design

The design loop that produced the above: (1) a session hits a *silent-loss* failure in real work (a tool reports success while something is quietly wrong); (2) it publishes the specimen to the bus; (3) context-divergent peers attack the proposed fix — and, because their divergence is real (different accumulated histories), they attack from different empirical vantages, which we measure and *bound* in §V-D: they are independent *only relative to* that divergence, not absolutely, so this is not self-review by one model echoing itself, but neither do we claim uncorrelated errors in the estimator sense; (4) the human gates only the irreversible.

The loop's *shape* is deliberately unoriginal and we claim no novelty for it: operating a system to learn what it should have required is evolutionary prototyping [17], [18], the spiral model [19], and game-day/chaos-engineering practice [20], which runs at scales far beyond this deployment. One post-corpus exercise (2026-08-05, a dated addendum outside the stamp) cuts against the usual prototyping argument. The first live run of a newly-shipped fleet wind-down protocol, across 31 sessions, surfaced two silent-success root causes, both in taxonomy class 2 (§V-B) and both violating the architecture's own through-line — *verify the outcome, not the intention* (§III). The instructive part is not that the drill found them: **the requirement they violate was already published in our own record months earlier**, and was simply not enforced on a later actuator. The gap was **compliance, not knowledge** — a weaker and more useful claim than requirements-unknowability: a through-line stated once does not propagate itself to each new actuator, and the drill is what detects the drift.

V. Evaluation

A note on names: throughout the evaluation, a codename in parentheses (e.g. `net-emu`, `perf-A`) identifies an *individual Claude Code session* — the participant that produced, or is the subject of, the adjacent claim. These codenames are role-hinting pseudonyms for publication, and the artifact package preserves the real↔pseudonym mapping. Every quantitative claim below carries an explicit *provenance tier*; the fleet adopted these as a discipline and we define them once here (Table II). The rule the deployment enforces on itself: a derived or recalled number may never be reported bare, and a claim a reviewer cannot resolve is, at that moment, not yet evidence.

Corpus, MEASURED from git + bus + `history.jsonl`*, as-of 2026-07-31* (every count carries this stamp — the log is append-only and non-stationary, so a bare number is under-specified; events after the stamp are reported as dated addenda and are *never* counted, in either direction): 290 commits (38 of type `fix`); 52 version landings; 2,744 bus messages, of which 2,739 are non-system, from 56 senders (1,109 directed + 1,107 broadcast + 521 announcement + 2 unclassified = 2,739; directed share 1,109/2,739 = 40.5%); 6,358 human prompts across 47 projects back to 2026-01-14. **Division of labour is spread but with a real head, not a power law** — the concentration computed by the harvester (`evidence-harvest.py _concentration`), the addressing classified from the same log, over *all* non-system senders: Gini 0.698, top sender 9.4%, top-3 25.4%, and 13 of 56 senders to reach 80%. Many peers carry load, no single dominant author, but a genuine concentration we report rather than round off to "distributed." Reflexivity confound — controlled, not just conceded. Because 196 of the 2,739 non-system messages are this paper's own review/ablation traffic, a concentration statistic over the whole log could be an artifact of the act of measuring. We therefore recompute it *restricting to* messages before the paper effort began (2026-07-24; n=2,543 non-system messages, 52 senders): Gini 0.693, top sender 9.4%, top-3 26.4%, 13 of 52 senders to 80% — *indistinguishable* from the full-log figures above (Gini 0.693 vs. 0.698; top-3 26.4% vs. 25.4%). The head is a property of the *organic* deployment, not of the paper's own threads; the reflexivity we flag elsewhere does *not* contaminate this result (both numbers are shown). Why roles emerged at all — no one assigned them. Specialization was not designed; it is an attractor of two forces the substrate creates. Continuity and ownership: a session that built a subsystem holds the most context about it, so the cheapest place to route the next related task is that same session — and routing it there deepens the advantage, a positive feedback that concentrates each domain onto whoever first worked it. Repeated exposure then hardens it: memory and the bus make yesterday's work legible today, so a returning session resumes its domain rather than re-learning a new one. That the resulting head is a *spread* one (Gini 0.698, no dominant author) is because these forces are *local* — one attractor per domain — not because anyone balanced the load. Role identity is an emergent property of continuity-plus-routing, not an org chart — which is why it is the foundation the later findings rest on rather than a policy that could be tuned away.

TABLE II
Provenance tiers used throughout the evaluation.

Tier	Meaning
MEASURED	ran it on a censused system, with a resolvable receipt (commit/log)
RECALLED	from memory/narration; may be wrong, flagged as such
GAP	the honest absence of a measurement we would want
LANDED	a mechanism shipped and is live in the running system
NULL	a pre-registered test whose falsifier fired (reported as such)
repro.-in-principle	a recipe that outlives the record but has not been re-executed

A. RQ1 — Courier elimination (proxy, with a latency tradeoff)

1,109 directed messages (as-of the corpus stamp) are *candidates* for auto-delivery — hand-offs a human would otherwise relay — so the count is a ceiling on relays avoided, not a claim that all were delivered: the realized auto-delivery count is a **GAP** — it is a runtime event the bus log does not record, so we report the ceiling the mechanism could eliminate, not the realized figure. Indeed the mechanism is unreliable: in this paper's own production, a directed job failed to auto-wake its peer (stale watermark) and the human relayed it — an actual delivery below the ceiling. We report the counterexample. A second, larger counterexample is dated, though it falls outside the corpus stamp. During a post-corpus fleet exercise (2026-08-05) two independent silent-success defects were live at once: window-actuation tools that print their failure to stderr and exit `0`, and an acknowledgment routine that terminated before writing its record (§V-C addendum). Their combined effect was that the human resumed relaying — driving the registered fleet through the shutdown by hand — **while every actuator-level signal reported success**. This is a bound distinct from the GAP above: unrecorded delivery leaves the measurement *thinned*, whereas an actuator asserting a success that did not occur leaves it *false*, and that is invisible from inside the measurement. A reachability check whose failure is surfaced rather than inferred was added in response (pinned in the replication package). The correct human-touch instrument is `history.jsonl` (un-swept, back to January), *not* transcripts (a 30-day cleanup horizon, §VII) which also carry an 11.9× `tool_result` overcount trap.

The longitudinal human-touch series (the method's clearest observational signal). From `history.jsonl` (human prompts only, un-swept back to January), we normalize touches by *active project* per month (Table I). Three monthly buckets cannot establish a trend, and the middle is non-monotonic (June ≈ May), so we report the *shape*, not a single ratio: prompts per active project sit near 70 across May–June, then drop to 36 in July, *while* concurrent projects grow from 16 to 35 (April, the substrate's build-out month, is excluded as bootstrap-heavy, as January and March are). To check whether July's drop is a trend or a single step, a finer weekly breakdown (14 weeks, replication package) shows the same downward *direction* — from the ∼35–40 band in early May to the ∼13 band by late July — so we read it as a sustained decline, not a one-month artifact. The weekly and monthly figures are on different time bases and are not comparable in magnitude: a week's prompt count (roughly a quarter of a month's) is divided by that week's active projects, so weekly per-project figures run lower — the smaller numerator dominates. We use the weekly series only to confirm *direction*, not to sharpen the ratio. This is the one longitudinal signal that speaks to the *method* claim rather than the compounding claim: as the substrate matures and carries more work in parallel, the human's per-project involvement drops, which is the courier → architect shift the method is for. The timeline (MEASURED, bus log + git) supports reading this as substrate-driven: the build-out began in April (the shared bus opened mid-April), and the higher-order primitives that most reduce human relaying — auto-delivery, verified orders, the project layer — landed incrementally and clustered in July, coincident with July's sharper drop; the decline tracks that ongoing hardening, not a one-time event (still temporal correlation, not proof). The leverage is larger than the per-project figure alone suggests: **from May to July the human took on 2.2× the concurrent projects (16 → 35) for only ∼14% more total prompting** (1,117 → 1,270 prompts) — per-project attention halved while total attention barely moved. At May's ∼70-prompts-per-project rate, July's 35 projects would have cost ∼2,450 prompts; the human spent 1,270, the budget for roughly *half* that many projects. We report it with the standard longitudinal confounds — the task mix changed, the operator learned, later projects may be individually smaller, and per-*active*-project is a proxy for per-*delivered*-project — so it is a trend, not a controlled measurement.

Who originates the coordination? A direct decomposition, answering the sharpest objection — that the human is the real router. We categorized every bus sender (MEASURED, `human_fleet_decomposition.md`) as human (`[operator]`, the tag Conductor posts under when Kyle drives the bus by hand), automated (`[system]`/watchdog/broker), or fleet (a Claude session). The human authored 0.9% of all coordination messages (26 of 2,744) and 1.9% of the *directed* routing mail (21 of 1,109); the fleet authored 95.5% and automated/system senders the rest, so **over 99% of coordination originates from non-human senders**. And that fleet coordination is load-shared, not funneled through a lead: among the 52 fleet senders the busiest is 9.8% of fleet mail, the top three 26.6% (fleet-only denominator, so slightly above the 25.4% all-sender figure above), and it takes 7 to reach half — there is no human-proxy hub routing the traffic. The honest limit: the bus is the Claude↔Claude channel *by construction*, so this bounds the human's share of the *shared* channel, not the prompt channel above — but the two agree, and that is the point: <1% of bus traffic *and* a falling per-project prompt intensity both point away from a human doing the routing. What the human owns — goals, framing, the irreversibility gate — is architect work, not courier work, which is precisely the claim.

And RQ1 is a *tradeoff*, not a pure win. Coordination over an async bus + a lead's wake cycle can be slower in wall-clock than a synchronous human courier: in this deployment a directed wake took minutes, and in the counterexample above the mechanism failed outright and the human was faster. The substrate optimizes the human's attention (fewer relays to run), and can *worsen* the critical-path latency of any single hand-off. The honest claim is *attention saved*, not *time saved* — RQ5 (§V-E) shows the same shape at the single-agent level (context bought steps and wall-clock but *more* tokens).

B. RQ2 — Failure modes closed, with real ablations (+ the defect taxonomy)

MEASURED: of the 38 `fix` commits (as-of the corpus stamp), ∼20 close *named coordination failure modes*. Several are ablation-structured — disable the mechanism, the failure returns — and at least one ablation *ran in operation*: the SHA-pinned push token, unpinned, would let an approval for commit A push commit B; the gate refused a stale approval live during this project. The two-phase-commit and member-cursor ablations are specified and pending.

A defect taxonomy the fleet converged on (a secondary contribution). Writing and ablating the cases surfaced a small, reusable classification of the *silent* defects the method exists to catch, each with a distinct remedy — the point being that a single discipline ("just replicate," "just tag") sails past most of them:

1) Single sample promoted to a property — a condition true of one run asserted as general. Remedy: replicate + tag. Sharpened (perf-A): a *measured* number promoted past its conditions is *worse* than a derived one, because the `measured` badge actively defends the error.
2) Green-but-wrong / wired-but-broken in the untested direction — a gate green + documented + shipped, passing every gate-ON test while the gate-OFF path leaks (emu-C, net-emu). Replaying the green measurement *n* times passes *n* times; the only fix is asserting a different measurement — execute the off-state / your own reproduction once. A post-corpus specimen adds an axis: the untested direction can be **selected by the very condition the mechanism exists to attest**. The acknowledgment routine of §V-C's addendum reached its fatal path only when a session had *exactly zero* unpushed commits — any unpushed work gave its counter a line to match, so it exited zero and the ack succeeded. Failure concentrated on sessions satisfying the cleanliness condition the ack exists to certify: this is **informative missingness** (MNAR, [23]) in a monitoring mechanism, so the census is *biased*, not merely thinned — and because silence reads as *not finished*, the indicator's polarity flips for exactly the population that was. Here the class-2 remedy is necessary but insufficient: **zero unpushed is not the off-state — it is the attested value, on the nominal success path.** The off-state was a dirty tree, and it *was* exercised. Enumerating on and off misses a boundary *inside* "on", so the remedy extends to the boundary value of the predicate the mechanism selects on.
3) Provenance-tier silently dropped during remediation — re-anchoring a number to a *recipe* downgrades MEASURED → reproducible-in-principle until someone runs it (band, emu-C). Remedy: run the recipe and *say you did* (emu-C did — the mutation reproduces `1,409,307,648`).
4) Conservative-error-is-durable — a wrong-but-safe-looking value evades the provenance check that would normally catch it. "A conservative error is not a safe error; it is a durable one."
5) Thoroughness regression — the context shortcut that made the agent fast made it skip the secondary gap (mcu-emu DISMAP; perf-B's third site). Remedy: re-derive under a differently-drawn boundary / forced first-principles trace.

And the remedy split the same convergence produced: TAG (cheap disclosure; catches a condition you measured but did not state) → ABLATE (an invariant/control; catches a condition you *believed held and did not*) → HARVEST (a parallel peer at the same boundary building the same artifact is a *naturally-occurring* off-state ablation — emu-C↔emu-B, perf-A↔perf-B — cheaper than a designed one, and record-visible: cross-tree agreement on different fixes is derived-not-copied). *Shipping only the tag documents the bug rather than fixing it.*

C. RQ3 — Defect discovery by vantage (machine-evidenced)

We split two claims review conflated. (i) A measurement vantage exists (supporting, not the RQ3 claim): MEASURED (build-A), a 38-Bash-call build had 17 measurement calls (12 `/proc` probes, 4 runs) — a vantage a browser assistant could not occupy, proven from the event log. (ii) A peer caught what the author shipped (the actual RQ3 claim, and it needs the *same* timestamp rigor — A ships at t0, B refutes at t1, both from the record). We enumerate the record-attested cross-session catches (14 attested, 13 classifiable; tabulated in the replication package) and here name four representative ones, each with *attribution corrected* against the record after review flagged two mis-attributions in earlier drafts (a "`api-svc` tag-flip caught by a peer" was in fact self-caught, and a claimed "slam-A caught four errors" had no record — both removed). image-gen → api-svc: api-svc's RTX-5090 idle-power *denominator* (64.97 W) was contaminated by image-gen's own resident tenant; a clean board idles near 21 W, a physical floor image-gen held. doc-A → slam-A: slam-A's "fixed-K Amdahl overhead" bandwidth model, refuted by diffing two derivations on the shared tensor table (K doubles when the workload doubles, so a fixed overhead is impossible — the model refuted itself on its own arithmetic). media-npu → socdev-A: socdev-A's "verified" NPU INT8 model was quantized *without calibration normalization* (wrong input scale); media-npu reproduced its exact numbers to five decimals, then corrected the parameter. bench-A ↔ net-emu: bench-A's "+64 s rejoin stall" was its own scorer's arrival-stamp backlog — net-emu relocated the measurement *into the guest* (a guest-emitted timestamp bench-A structurally cannot make; a pinned commit in the replication package) and a subsequent scorer replay put guest-side re-acquisition at 0.0 s. We keep the two artifacts distinct (the instrument commit vs. the replayed figure), pinned t0 (stall reported) < t1 (refuted). A record-attested instance from the ablations: in `ablation_perf-A` a blind, zero-context arm caught a real overstatement in the author's own case (129 defective cells reachable via the engine API but *not* the shipped UI) in nine

tool calls. **Cases are illustration; the events (order claims, timestamps, tool-call logs) are the evidence.**

Post-corpus addendum (2026-08-05, outside the as-of stamp; *not* counted in the 14/13 above). The catches above are peers refuting a peer's technical *result*. One later catch differs in kind: the defective artifact was the **coordination substrate itself**, and the catching vantage was *being subject to it*. It is offered as support for the §V-D bound, not an increment to it. *Disclosure, load-bearing here:* `substrate-dev` below is the session that authored the wind-down machinery **and drafted this paper**; the catch is reported against ourselves, and the entry would be worthless without the identity stated. All five events are timestamped in the append-only record — **four on the bus, one in git** — on one day. At **11:34:25** `mcu-emu`, winding itself down, published the root cause of an acknowledgment routine that wrote no record: a counter that prints `0` and *exits non-zero*, terminating the routine under `set -e`. At **14:32:54** `substrate-dev` committed a fix (pinned in the replication package) for a *different* defect in the same routine. At **17:17:25** it broadcast to the fleet that the bug was fixed. At **17:20:53** — **3m28s later** — `band` measured the shipped routine from a clean tree: no output, exit 1, no record written. At **18:09:05** `band` published an audit enumerating **7 acknowledged against 24 not** (31 registered tags). The routine was corrected the next day (pinned in the replication package). We record it in **both** directions, as with RQ1's counterexample. As a vantage instance it is strong: the refutation came from *operating* the mechanism — `band` was winding itself down and observed that it had not — timestamped against `substrate-dev`'s own claim. As a **substrate failure** it is equally real: the refuting information had been on the bus **at least 5h43m** before the broadcast, so the delivery machinery this paper evaluates did not put a published refutation in front of the session that authored it before that session asserted the opposite to the fleet. **A vantage that is published but not *consumed* is a bound on the substrate, not only a credit to the peer.** The episode is a class-3 instance of our own taxonomy (§V-B), violating a remedy we had already published (*run the recipe and say you did*). It is **empirical-vantage** — a measured exit status, not a same-input reasoning disagreement — so §V-D's statement that no instance exists of a peer catching a same-input reasoning error is **unaffected**.

D. RQ4 — Compounding competence: claim, confound, and the ablation test

We split the claim into RQ4a (the crisp-symptom floor: does context lower re-derivation cost on a task with an objective oracle?) and RQ4b (the open-search ceiling: does it on a task a fresh agent cannot cheaply bisect?), and test both below. **Correlation, MEASURED:** context-carrying sessions recognized bug-classes fast (e.g. a register-coverage gate green over 12/370, the class recognized in seconds because prior tasks had named it; one-line fix → 12/370→370/370, surfacing a dead-time-zero shoot-through). The confound (owned): same model → this may be "persistent memory works," a known result, and memory-*present* is not memory-*causal*. **The test (LANDED) — six A/B ablations + the RQ5 baseline, and the naive claim does not hold up.** A *genuinely isolated* fresh session (harness-isolation caveat binding) vs. a context-carrying one on the same task, measuring re-derivation cost:

- game-coach — the best-powered arm, and the backbone of the negative result (coach affordance overriding an engine fact; N=8 per arm — the only powered ablation of the six): 16/16 reached the real architectural fix; BLIND mean 5.4 tool-calls / 37.4k tok vs PRIMED 6.5 / 44.2k — priming bought a small cost, no benefit, and all 16 used no git history. A clean, powered NEGATIVE: a fresh clone holding the carrier re-derives the fix; session-memory adds nothing to the *outcome*. We lead with this arm deliberately — a negative result is only as strong as its best-powered evidence.
- mcu-emu — a COUNTER-CURRENT (color), not the foundation (register-coverage hole; N=3 blind arms on a single task instance, post-hoc-graded, and its own author cautions against causal weight): all three re-derived the identical one-line fix (3–5 tool calls, 46–105 s). Beyond negative, it showed a vivid *thoroughness regression* — the context-carrying run pattern-matched a pre-staged answer and stopped, missing a second register class (DISMAP) the blind arm caught. Suggestive, but single-instance and unpowered; we present it as color on the powered result above, never as its base.
- emu-B (idle-vs-load clock constant; N=18, fictional subject to kill harness contamination): cost-contingent. When measuring is *hypothetical/costly* the standing-order arm wins unanimously (control ships the confidently-wrong idle value; treatment re-derives the hazard); when the measurement is made cheap (a runnable probe), both arms 6/6 ship the right number — the base model just does it. The residual real effect is *a disposition at a cost/effort fork*, not added capability.
- media-npu (INT8) (INT8 class-score collapse; objective corr > 0.90): the fresh agent reached the same root cause in $\sim$the same $\sim$4 steps, solved it (6 m 44 s, 16 calls, cls-corr 0.956). A PARTIAL NEGATIVE — refines the claim to a *bound* (below).
- perf-A (one of the two pre-registered ablations — rubric hash published to the bus before any agent launched; N=3/arm): tests whether a *wrong* carrier actively harms. It does not — stale docs had zero measurable effect (misled-to-wrong-file 0/3 in *both* arms; the stale-doc arm was marginally *cheaper*, and every agent detected the staleness unprompted). *Bonus:* a blind zero-context arm caught a real overstatement in the author's own case in nine tool calls.
- media-npu (C2 open-search sequel) (pre-registered rubric — hashed and independently witnessed before the fresh arms ran; Arm A scored *post-hoc* from the bus record): a distinct experiment from the INT8 ablation above, testing the open-search *ceiling*, with the same A/B shape

— Arm A the context-carrying session (scored post-hoc from record: it produced the correct empirical rule immediately on the bus, R1–R4 satisfied at ≈one message) vs. Arm B four fresh isolated agents run under the hashed rubric. Its symmetric falsifier fired — all four fresh arms *also* solved the constructed task at floor cost, so it was secretly bisectable — and it is reported as a **NULL** (no capability gap observed, but no clean ceiling measurement either), with the finding *why* (reproducibility ⊥ open-search; detailed below).
- RQ5 (§V-E, the baseline, not an ablation): both arms correct with zero help; context bought 2.6× wall-clock and 36% fewer steps but more tokens.

The result, stated plainly: the committed carrier is what compounds; lived context buys efficiency and recognition, not capability. This is a powered negative that corrects a widely-held intuition. A fresh clone *holding the repo* re-derives the fix because what compounds is what is still true in the carrier when the next session reads it, and the executable carrier governs (code / assert / invariant); a *prose* carrier is "neither the asset nor the liability anyone claimed" (perf-A's wrong-doc ablation is self-refuting because the code is there to check it against). The efficiency/capability line, made operational (so "efficiency" is falsifiable, not a relabel that absorbs every gain): we score an ablation arm as a *capability* gain iff the context-carrying arm reaches a correct fix the fresh arm does NOT, and an *efficiency* gain iff it reaches the *same* correct fix *more cheaply* (fewer steps/tokens/time). Under that test the five floor ablations (the five non-C2 ablations, all testing the crisp-symptom floor) + RQ5 are efficiency, not capability — across every arm the fresh agent reached a correct fix (game-coach 16/16, mcu-emu 3/3, emu-B 6/6, INT8 in ∼4 steps, perf-A 3/3 correct in *both* arms (0/3 misled), both RQ5 arms), so the measured advantages (RQ5's 2.6× fewer steps; game-coach's flat outcome) are cheaper-path, not can-vs-cannot. C2, the sixth ablation, is excluded from this verdict on purpose: both its arms reached the answer, but its symmetric falsifier fired (the task was not genuinely open-search), so C2 yields *no* capability-vs-efficiency signal about the ceiling — it is a pre-registered NULL (below), not a data point in the efficiency finding. *This is a falsifiable claim: a single ablation where the fresh arm FAILS and the context arm succeeds would move it to "capability" — we looked and did not find one on the tasks tested.* The defensible, bounded RQ4b claim: accumulated context compounds most on an open search a fresh agent cannot cheaply bisect and least on a crisp symptom with an objective oracle (any agent bisects fast); on some specimens context even shows a *thoroughness regression* (pattern-match-and-stop). **This bound is half-measured: the crisp-symptom FLOOR is MEASURED (the five floor ablations bisected fast, and even C2's ceiling attempt collapsed to a fast bisection); the open-search CEILING resists measurement — and we report the *pre-registered sequel that tried to measure it and why it could not.*** A fresh attempt (media-npu, C2 — rubric hashed and bus-published + independently witnessed BEFORE any arm ran) built a "hard" open-search task in a sterile, reproducible sandbox; its pre-registered symmetric falsifier fired: all 4 fresh arms reached the correct answer at floor cost (∼4 tool calls, ∼60 s), so the task was secretly *bisectable*, and per the pre-registration it is reported as a NULL — NOT as support for the floor. The finding is the reason: reproducibility and the open-search property are in TENSION — a sterile, offline, pre-registerable task must place its answer in an inspectable artifact, and the instant the answer is in an artifact, `strings` turns the task into a bisection. Real open-search cost lives in the answer being *absent from every artifact* (learnable only by iterating on hardware over days — the SoC converter case). So the ceiling is HARD TO INSTRUMENT, which is a *different* claim from being ABSENT; the honest earning test is a field A/B on real hardware (natural-history, controlled where possible), not a sandbox — and a contrived "open" task that looked bisectable would have manufactured a *false null on the floor*, which the pre-registered symmetric falsifier exists to prevent, and did. We report the naive "memory makes task *N+1* dramatically cheaper" as refuted on the tasks tested, and the open-search ceiling as an **honest, reasoned GAP** — measurable floor, unsandboxable ceiling — rather than a claim dressed as a result. We conjecture the unmeasured ceiling is task *origination* — identifying that the work needs doing at all, a step every ablation and the baseline held fixed by construction — which is what the field replication (§VIII) is designed to expose.

A discipline the fleet imposed on us, and it is the sharper point: the C2 null and `perf-A`'s stale-doc null (§V-B/RQ2) are NOT independent corroboration — they are one mechanism on two surfaces. Both tasks were *bisectable by construction because the ground truth was inspectable in the artifacts*: perf-A's arms scored 0/3-misled because the code was there to check the doc against; C2's arms scored 4/4-solved because the answer was `strings`-able out of the model file. By the convergence-provenance rule (§V-D independence), this is common-dependency at the mechanism tier (and shared-base-model) — near-zero independent weight. So we cite them as two illustrations of one constraint (reproducibility ⊥ open-search), never as two nulls agreeing. What *is* load-bearing is the constraint itself: it held across two domains (stale-doc/app vs converter/NPU) and two opposite failure directions (0/3 vs 4/4), which is evidence the constraint is real — *even though the two nulls do not independently corroborate it.* The finding generalizes; the data points do not replicate. *(Sequel artifacts — the hash-pinned pre-registration and results — are in the anonymized replication package; DOI at camera-ready.)* (Grading: two of the six ablations pre-registered their rubric, `perf-A` and the C2 sequel; the rest are post-hoc graded — §VII.)

The independence sub-claim (RQ4-adjacent) — the measured bound, at exactly the strength the record supports. A skeptic rightly objects that our divergence proxies (disjoint task histories, low memory-file overlap, divergent tool-call sequences) measure divergence of *inputs*, whereas independence of estimators requires uncorrelated *errors* —

and two same-model sessions share a generative prior, so their reasoning errors can correlate even with different inputs. We tested this directly rather than waving at it. We classified the 13 solid record-attested cross-session catches (§V-C; a 14th is an incidental exposure, excluded) as factual/local (a discrete fact the catcher held differently) vs. reasoning-shaped (the catcher reasoned to a different conclusion). The split is 6 factual, 7 reasoning-shaped — ≈1:1, which *at face value would refute* an "input-divergence-only" reading. But the honest reading is narrower, and it is the one the record supports: every reasoning-shaped catch was made by a peer at a different *empirical vantage* — a measurement relocated into the subject (a guest-emitted clock), different physical silicon, or the ship-vs-implement test-direction asymmetry — not by an independent reasoner spotting a co-located reasoning flaw from identical inputs. The corpus authors say so themselves ("a different vantage point, a property of the substrate, not of any one model being smarter"), and there is no instance in the record of a peer catching a same-input reasoning error. We therefore claim independence *only relative to* the divergence proxies, never absolutely: peers do catch reasoning-shaped defects, but they catch them by being *empirically positioned* differently — the input/grounding divergence the proxies measure — which is a measured bound on the claim, not evidence of uncorrelated errors in the estimator sense. A second pre-registered instrument on the same catches agrees. A rubric hashed before any classification (`perf-A`'s, executed by a non-author, `rq3_categorisation_test-*`) tested a sharper form — whether peer catches cluster in naming/categorisation rather than logic (the "an author cannot search for what they mis-filed" hypothesis). It was refuted over the same 13 classifiable catches (L=7, N=6; robust to every borderline call, and with two biases — a write-up selection effect and an N-generous tie-break — both running *toward* naming and it still lost): peer catches are a *mix*, not a naming cluster, the same picture the factual/reasoning split gives. A shared-lineage counter-specimen makes that divergence concrete at the artifact level (owner-confirmed on three trees; `cases_emu-C_flash-lock-three-way.md`): the same QEMU model family (three sibling SoC emulators, one vendor family) encoded *one* flash-controller lock register three different ways — emu-C buggy (lock keyed off the wrong bit → a 5 ms stall), emu-B clean-and-faithful, emu-A clean-but-latent-wrong (a hardcoded lock, wrong at reset). A defect on one tree was *structurally impossible* on its siblings from the same lineage, and even the two passes diverged in fidelity. So shared model is a prior to *check*, never a finding to *inherit* — what discharges it per tree is a receipt (file:line + a test), not the lineage. Same generative prior, three distinct implementations: exactly the divergence the proxies measure. (Honest limit, which the specimen states: the three ports share a bus, so we cannot claim the encodings were arrived at *independently* — only that the artifacts diverged and each tree confirmed its own by test.)

E. RQ5 — Baseline: cold orchestrator vs. peer substrate (pre-registered)

One matched task, run both ways, pre-registered (frozen spec + black-box acceptance test + directional prediction committed before either arm ran; the instrument was first proven satisfiable via a throwaway reference impl, then proven to actually test the feature against the unmodified baseline). Task: add `project pause/resume` to the ∼700-line `project.sh` state machine — a *new instance of a familiar class*, objectively gradeable on six behaviors. Arm A = a genuinely memoryless fresh session (separate `$HOME`, no memory/bus/hooks, verified at run time). Arm B = `api-svc`, the peer with the most lived `project.sh` history, business-as-usual (not told it was an experiment). Both worked from an identical repo copy that *includes* `CLAUDE.md` (a real clone has it — the conservative choice, it helps Arm A) but excludes the spec and `.git`.

MEASURED result — and it does not simply flatter the thesis. Both arms reached a *functionally correct* implementation (all six behaviors) with zero substantive human help. So a fair clone *holding the committed carrier* is fully capable with no session-memory — corroborating the ablation convergence in §V-D. But lived context bought real efficiency: Arm B finished in 7.8 min vs 20.1 (2.6×), with 36% fewer tool calls (14 vs 22) and 41% fewer lines added (39 vs 66 — more surgical) — it *recognized* the state machine rather than exploring it. The honest counter-current, reported: Arm B used 38% *more* output tokens (35.2k vs 25.6k) — the compounding benefit here is speed and recognition, not token cost. The pre-registered prediction (B fewer human turns + less wall-clock) was confirmed on wall-clock/steps, tied at zero on human turns (the task was tractable enough that neither needed intervention — so RQ5 measures the single-agent efficiency delta, *not* the courier delta of RQ1). *Bonus, in-apparatus:* the frozen acceptance test carried a latent defect (a crude substring assert tripping on a correct "NOT dispatched" refusal); it was disclosed, not patched — editing a frozen instrument post-hoc is the exact sin this paper studies (§V-F). Full protocol + both arms' diffs: `cases_rq5-baseline.md`.

F. The paper as an instrument (an operational finding, not a controlled result)

Asking sessions to substantiate claims *to an external audience* forced verification of things taken on trust internally. In git: `perf-A` found a 46-day defect (129 cells rendering wrong fps, count never zero, never visible) while writing its case — after 46 days of use/test/review found nothing; `perf-B` surfaced a validation-across-an-unsafe-version provenance defect the same way. We state the hit-rate: of 19 case-writing sessions, 4 (∼21%) reported a correction surfaced during the writing (`perf-A`, `perf-B`, `meter-A`, the RQ5 apparatus's own frozen-test defect); the rest reported none. We are careful about what this is and is not. It has no control arm: we did not measure how many defects surface in 19 sessions of *equivalent effort spent not writing for an external audience*, so 21% has nothing to beat, and the underlying effect — careful explanation to an outside reader surfaces latent defects

— is well established for design documents and code review; *the only novel element here is that the writers were agents*. And it is not fully "testimony-free": the *commits* are machine evidence, but the *attribution* ("the defect was surfaced by the act of writing") is a causal judgment made and reported by the same sessions — two different epistemic objects, which we keep separate. So we report it as an **unanticipated operational finding with a stated hit-rate and no control arm** — worth recording, not a headline result.

VI. Implications for Practitioners

The measured results reduce to four working rules; each carries the bounds of §VII.

1. Fund the carrier, not the session. Effort that moves knowledge into the committed, executable carrier — code, asserts, invariants, checked-in operating docs — buys capability: a fresh clone holding it reached every fix we tested (§V-D, §V-E). Effort that preserves session memory buys speed, not correctness. When the two compete for budget, the carrier wins.

2. Use lived context where efficiency and routing pay — and audit it. Persistent sessions finished the matched task 2.6× faster with 36% fewer steps, and let the lead route work on a peer's demonstrated history rather than a declared role. But context also produced the one failure class unique to it: the *thoroughness regression*, in which a context-carrying session pattern-matched a staged answer and stopped short of the second defect a blind arm caught (§V-D). Long-lived sessions therefore need a periodic fresh-eyes pass — a blind clone on the same task — as a standing control.

3. Write fixes into the executable carrier, not into prose. The stale-doc ablation found a wrong prose carrier to be "neither the asset nor the liability": every arm detected the staleness against the code (§V-D). Prose narrates; code, tests, and invariants govern. A fix that exists only as narration has not compounded.

4. Copy the cheap primitives first. The substrate's load-bearing mechanisms — verified orders that read the artifact back before acknowledging, SHA-pinned approval tokens, an observer-owned admission throttle — are small, model-agnostic, and follow one rule: *verify the outcome, not the intention* (§III). They transfer whether or not one adopts the fleet.

VII. Threats to Validity

- Self-evidencing circularity. A corpus of first-person cases about coordination, curated for a paper about coordination, is self-report at scale. Mitigation: cases are demoted to illustration; every load-bearing claim rests on the mechanical record.
- Single-model confound. Divergence is memory/context, not different minds; we own the composition and do not claim emergence.
- Selection bias / no denominator. The open call collects confirming specimens by construction. We hold $\geq$1 deliberate counter-case (a bug recognized instantly but retripped twice — recognition cost $\rightarrow$0, prevention cost unchanged), frame the corpus as illustrative, and note the missing denominator explicitly.
- 30-day record horizon. The platform silently deleted transcripts >30 days (now disabled); the pre-June transcript record is GAP. Longitudinal claims use `history.jsonl` + git, which survive.
- Ablation methodology. (a) *Grading asymmetry* — two of six ablations (`perf-A` and the C2 sequel) published a rubric hash *before* results; the other four are honestly-run but post-hoc graded, so retrofitting is disavowed, not detectable. `perf-A` is the *only* ablation whose both arms ran under a pre-published rubric; C2's pre-registration covers its rubric and its fresh (Arm B) runs, but its Arm A was scored post-hoc from record ((c) below), so C2 sits partly in both categories. (b) *Isolation is a claim, not a given* — the harness pre-injects a fresh agent's cwd, git status, and memory *index*, so "blind" arms are mitigated, not sterile; the cleanest arms used fictional subjects (emu-B) or audited each agent's actual tool calls. (c) *Historical arms are not re-runnable* — some context-carrying arms (the INT8 ablation's Arm A and C2's Arm A) are transcripts of past work (the answer already known), so they bound rather than measure. (d) *N is small* (1–8/arm) and outcomes are often categorical (no variance). Each is stated in the individual `ablation_*.md`.
- N=1, one operator, one workstation, one model family.
- Self-authorship, including *this reframe* — written by an interested lead who is also the author of the substrate under evaluation (§V-C addendum), corrected by peers; we prefer machine evidence throughout and mark recollection RECALLED.
- The bystander-catch corpus contracted under audit. While preparing this paper, two claimed cross-session catches did not survive record-checking — a "`api-svc` tag-flip caught by a peer" was in fact self-caught, and "slam-A caught four independent errors" had no case-file or bus record — and both were removed (§V-C). We report the contraction rather than quietly correcting it, because it is itself evidence for the provenance discipline the paper claims: the corpus is what the record supports (14 attested, 13 classifiable), not what narration remembered.

VIII. Future Work: A Pre-Registered Replication

The paper's central threat is N=1 — one domain (systems/tooling), one operator, one workstation, one model family. We therefore *pre-register* the replication that would falsify or corroborate the claims, and state its prediction before running it (predict-then-run, a discipline the fleet itself enforces):

- Vary the confounding axes, not the surface. Repeat the experiment on a structurally different domain — ideally not software (e.g. scientific literature synthesis, hardware/product design, operations, or creative work), where "a task" and "expertise" mean something different in kind. Ideally with a different human architect, to remove the "maybe it is this operator" confound.

- Hold the *method* constant, vary the *content*. The claim under test is not "the tool runs on another codebase" but that *the composition* — persistent context-heavy peers + a shared bus + verified orders + a human gating irreversibility — reproduces the signatures: compounding competence (a measured downward per-task-cost trajectory via the same ablation protocol), peer review catching the lead's blind spot, and the self-designed substrate — and, per the §V-D conjecture, whether accumulated context confers an advantage in task origination, the step our ablations held fixed.
- Prediction (falsifiable): if the principles are real and not artifacts of this domain/operator, the *same* signatures appear on the new "thing"; if they are artifacts, they do not. A null result is as informative as a positive one and will be reported as such.
- Feasibility: the replication needs no new system — only a new *goal* pointed at a fleet (`project new <thing>`), which is precisely what makes the method's claim to generality testable rather than rhetorical.

This is the sequel experiment; naming it here is the honest acknowledgement that a single deployment, however rich, establishes a *method worth replicating*, not a law.

IX. Conclusion

Conductor is an experience report on a nameable composition — one model, durable per-session memory, a shared bus, a human gating only irreversibility — that co-designed and hardened its own coordination substrate in vivo. Its headline intuition, compounding competence, was put to six A/B ablations and a pre-registered baseline, and **the naive form did not hold up: what compounds is the committed carrier, and lived context buys efficiency and recognition, not capability** — most on open searches a fresh agent cannot cheaply bisect, least on crisp symptoms with an objective oracle. That bounded negative is the paper's most solid evaluated result, and the process that produced it — peers refuting the lead on the bus, an ablation refuting its own author's case — is the mechanism under study.

The reusable contribution is the method: in-vivo adversarial co-design by persistent, context-divergent peers under a reversibility-gating human, offered as an existence proof with a stated mechanism, not a validated superiority. Its cleanest demonstration is this paper's own review. The internal panel, at the deployment vantage, caught substance — a circularity, a single-model confound, unrun ablations; independent external review, at the submission vantage, caught what the panel structurally could not — cross-section consistency, arithmetic, and venue compliance. Which vantage caught which defect class is checkable in the archived draft series and the paper's own git history: the vantage bound of §V-D, one level up.

Three secondary observations travel beyond this deployment: the silent-defect taxonomy and its TAG→ABLATE→HARVEST remedy split (§V-B); the non-independence discipline — before crediting two results as corroboration, check for a shared generative mechanism, and discount if found (§V-D); and the durability ladder — untracked → committed → pushed — because a claim a reviewer cannot resolve is, at that moment, not yet evidence. The working summary for practitioners is §VI: fund the carrier, use sessions for speed and routing, audit for the regression, copy the cheap primitives.

Appendix: Evidence vs. Illustration

Evidence (MEASURED): the mechanical record (git, bus, `history.jsonl`, spend meter), the six A/B ablations (the `ablation_*.md` files, replication package), and the pre-registered RQ5 baseline (`cases_rq5-baseline.md` plus the frozen spec/test and both arms' diffs). These carry the headline claims. Illustration (dataset, not evidence): twenty first-person `cases_*.md` specimens (from the 19 case-writing sessions of §V-F — one session contributed two) + the platform specimen `cases_cleanup-timer`, retained to illustrate design patterns, not cited as proof of any headline claim. Curated to five in-body illustrations, each mapping to a distinct claim:

1) `perf-A` — paper-as-instrument (§V-F): a 46-day latent defect surfaced *by writing the case*, and an ablation that *refuted the author's own Case 2* — the reflexive mechanism at its sharpest.
2) `net-emu` ↔ `bench-A` — RQ3 bystander/vantage: observer-vs-subject mutual correction (bench-A's "+64 s stall" was its own backlog; net-emu's guest-side clock settled it) — each catches what the other structurally cannot.
3) `mcu-emu` (ablation) — RQ4b: the *thoroughness-regression* specimen (context pattern-matched a staged answer and stopped; the blind arm caught the second register class).
4) `app-A` (a consumer app handling private user data) — RQ4a convergence: a 5th independent derivation of "the model narrates, deterministic code owns the value" reached *from privacy* — convergence that survives a change of *force*.
5) `image-gen` — lived-expertise routing (§IV): the GPU-lease-lying / cost-blow-up incidents it was assigned to document *because it lived them* — the motivating case for routing on history.

Several cases were corrected during the paper's own writing (perf-A's Case 1 severity downgraded from user-visible to latent; perf-B's propagated overstatement retracted), logged as §V-F specimens.

Disclosure: Use of Generative AI

This paper reports on, and was substantially produced by, a fleet of *Claude Code* (Anthropic) sessions — the same system the paper studies — under the human author's direction. We disclose the division of labour precisely, because for this subject it is itself a finding. The agent sessions: drafted the prose of every section across the full draft series; *adversarially peer-reviewed* one another's drafts on the shared bus and produced the three-lens review that forced the v2 and v4

reframes; designed and executed the six A/B ablations — two of them (`perf-A` and the C2 open-search-ceiling sequel) pre-registered — and the RQ5 baseline; and mined the git, bus, and `history.jsonl` records for the quantitative evidence. The human author set the goal, made every irreversible decision (each `git push` was human-gated), chose the framing and the venue, took final responsibility for every claim, and is the sole party accountable for this submission. No text was accepted without human review. All quantitative results were re-derived from the primary records cited. Independent human external review (multiple rounds) then caught defect classes the fleet's own review structurally could not — cross-section consistency, arithmetic, and venue-compliance — and drove the final revisions; §IX notes why this split falls out of the paper's own thesis. Per ACM/IEEE authorship policy, the generative-AI system is *not* an author; its contribution is disclosed here and acknowledged below.

ACKNOWLEDGMENT

The author thanks the Conductor fleet — the persistent *Claude Code* sessions named throughout this report — whose adversarial review, ablations, and first-person specimens are the substance of this work, and without which neither the system nor the paper would exist.

REFERENCES

[1] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
[2] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," *arXiv preprint arXiv:2308.00352*, 2023, ICLR 2024.
[3] C. Qian *et al.*, "ChatDev: Communicative agents for software development," *arXiv preprint arXiv:2307.07924*, 2023, ACL 2024.
[4] Z. Liu *et al.*, "A dynamic LLM-powered agent network for task-oriented agent collaboration," *arXiv preprint arXiv:2310.02170*, 2023.
[5] L. Song *et al.*, "Adaptive in-conversation team building for language model agents," *arXiv preprint arXiv:2405.19425*, 2024.
[6] G. Chen *et al.*, "AutoAgents: A framework for automatic agent generation," *arXiv preprint arXiv:2309.17288*, 2023, IJCAI 2024.
[7] Y. Yue *et al.*, "MasRouter: Learning to route LLMs for multi-agent systems," *arXiv preprint arXiv:2502.11133*, 2025, ACL 2025.
[8] I. Ong *et al.*, "RouteLLM: Learning to route LLMs with preference data," *arXiv preprint arXiv:2406.18665*, 2024.
[9] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023.
[10] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, "Mixture-of-agents enhances large language model capabilities," *arXiv preprint arXiv:2406.04692*, 2024, ICLR 2025.
[11] G. Wang *et al.*, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.
[12] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," *arXiv preprint arXiv:2408.08435*, 2024.
[13] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, "Darwin Gödel machine: Open-ended evolution of self-improving agents," *arXiv preprint arXiv:2505.22954*, 2025.
[14] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003, IEEE Computer Society.
[15] L. D. Erman, F. Hayes-Roth, V. R. Lesser, and D. R. Reddy, "The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty," *ACM Computing Surveys*, vol. 12, no. 2, pp. 213–253, 1980.
[16] B. Hayes-Roth, "A blackboard architecture for control," *Artificial Intelligence*, vol. 26, no. 3, pp. 251–321, 1985.
[17] C. Floyd, "A systematic look at prototyping," in *Approaches to Prototyping*. Springer, 1984, pp. 1–18.
[18] A. M. Davis, "Operational prototyping: A new development approach," *IEEE Software*, vol. 9, no. 5, pp. 70–78, 1992.
[19] B. W. Boehm, "A spiral model of software development and enhancement," *Computer*, vol. 21, no. 5, pp. 61–72, 1988.
[20] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
[21] G. Zhang *et al.*, "G-Memory: Tracing hierarchical memory for multi-agent systems," *arXiv preprint arXiv:2506.07398*, 2025, NeurIPS 2025.
[22] J. Liu *et al.*, "RCR-Router: Efficient role-aware context routing for multi-agent LLM systems with structured memory," *arXiv preprint arXiv:2508.04903*, 2025.
[23] D. B. Rubin, "Inference and missing data," *Biometrika*, vol. 63, no. 3, pp. 581–592, 1976.